

Guia Completo - CRUD de Fornecedores

FastAPI + PostgreSQL + Swagger/OpenAPI

Projeto Portal B2B de Comercializacao com Microservicos

Objetivo do material

Este guia foi escrito para que o aluno consiga iniciar do zero, instalar as ferramentas, criar o banco de dados, programar o backend, executar a API, testar no Swagger e validar os dados no PostgreSQL.

Ideia central do dominio

No modelo deste projeto, fornecedor nao e uma tabela isolada. Fornecedor e uma EMPRESA que possui o perfil FORNECEDOR. Por isso o CRUD manipula Empresa, Perfil, Empresa_Perfil e Endereco.

1. O que sera desenvolvido

Neste laboratorio sera desenvolvido um microservico de fornecedores. Ele sera usado como modelo para os demais microservicos do projeto. A arquitetura segue uma divisao simples: modelos representam tabelas, schemas validam dados de entrada e saida, rotas expõem os endpoints e o arquivo principal inicia a API.

- Cadastrar uma empresa fornecedora.
- Associar automaticamente o perfil FORNECEDOR a essa empresa.
- Cadastrar um endereco para a empresa.
- Listar fornecedores cadastrados.
- Buscar fornecedor por ID.
- Atualizar dados cadastrais.
- Inativar fornecedor sem apagar fisicamente do banco.

Importante

A exclusao sera logica. Em vez de remover a linha do banco, o sistema muda o status para INATIVA. Isso preserva historico, pedidos futuros e integridade dos dados.

2. Ferramentas necessarias

Ferramenta	Para que serve
Python 3.11 ou superior	Linguagem usada para desenvolver o backend.
PostgreSQL	Banco de dados relacional usado para armazenar as tabelas.
pgAdmin ou DBeaver	Interface grafica opcional para visualizar banco e tabelas.
VS Code	Editor recomendado para escrever o codigo.
Terminal ou PowerShell	Usado para criar ambiente virtual, instalar dependencias e executar o servidor.
Navegador	Usado para acessar o Swagger em http://localhost:8000/docs .

3. Instalacao do Python

Instale o Python a partir do site oficial. Durante a instalacao no Windows, marque a opcao Add Python to PATH. Sem isso o comando python pode nao funcionar no terminal.

3.1 Verificar instalacao

```
python --version
pip --version
```

Se os comandos retornarem versoes, a instalacao esta correta. Se aparecer mensagem de comando nao encontrado, o Python provavelmente nao foi adicionado ao PATH.

4. Instalacao do PostgreSQL

Instale o PostgreSQL e defina uma senha para o usuario postgres. Para este laboratorio usaremos a senha postgres, mas em projetos reais isso deve ser mais seguro.

4.1 Criar o banco de dados

Voce pode criar pelo pgAdmin ou pelo terminal psql. O nome do banco sera portal_b2b.

```
CREATE DATABASE portal_b2b;
```

Se estiver usando pgAdmin, clique com o botao direito em Databases, escolha Create -> Database, informe portal_b2b e salve.

5. Criar a pasta do projeto

Abra o terminal na pasta em que deseja guardar os codigos e execute:

```
mkdir fornecedor-service
cd fornecedor-service
```

Essa pasta sera o microservico de fornecedores. No futuro, os outros grupos podem criar pastas semelhantes, como produtos-service, demanda-service, pedidos-service etc.

6. Criar e ativar o ambiente virtual

O ambiente virtual separa as bibliotecas deste projeto das bibliotecas instaladas no computador. Isso evita conflitos entre projetos diferentes.

6.1 Windows

```
python -m venv venv
venv\Scripts\activate
```

6.2 Linux ou Mac

```
python3 -m venv venv
source venv/bin/activate
```

Quando o ambiente estiver ativo, normalmente aparece (venv) no inicio da linha do terminal.

7. Instalar dependencias

Com o ambiente virtual ativo, instale as bibliotecas:

```
pip install fastapi uvicorn sqlalchemy psycopg2-binary pydantic email-validator passlib[bcrypt]
```

Biblioteca	Funcao
fastapi	Framework para criar a API REST.
uvicorn	Servidor que executa a aplicacao FastAPI.
sqlalchemy	ORM para mapear classes Python para tabelas do banco.
psycopg2-binary	Driver de conexao com PostgreSQL.
pydantic	Validacao dos dados de entrada e saida.
email-validator	Permite validar campos do tipo EmailStr.
passlib[bcrypt]	Criptografia da senha antes de gravar no banco.

8. Criar estrutura de arquivos

Crie os arquivos abaixo dentro da pasta fornecedor-service.

```
fornecedor-service/
|-- main.py
|-- database.py
|-- models.py
|-- schemas.py
|-- fornecedor_routes.py
|-- requirements.txt
```

Cada arquivo tem uma responsabilidade. Isso deixa o codigo organizado e facil de replicar em outros microservicos.

Arquivo	Responsabilidade
database.py	Configura a conexao com o PostgreSQL.
models.py	Define as tabelas usando SQLAlchemy.

Arquivo	Responsabilidade
schemas.py	Define os modelos de entrada e resposta usando Pydantic.
fornecedor_routes.py	Contem os endpoints do CRUD.
main.py	Cria a aplicacao FastAPI, configura CORS e carrega as rotas.
requirements.txt	Lista as dependencias do projeto.

9. Criar o arquivo requirements.txt

Depois de instalar as dependencias, gere o arquivo requirements.txt:

```
pip freeze > requirements.txt
```

Esse arquivo permite que outro aluno instale as mesmas bibliotecas usando `pip install -r requirements.txt`.

10. Programar database.py

Este arquivo centraliza a conexao com o banco. Se o usuario, senha ou nome do banco forem diferentes, ajuste a variavel `DATABASE_URL`.

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base

DATABASE_URL = "postgresql://postgres:postgres@localhost:5432/portal_b2b"

engine = create_engine(DATABASE_URL)

SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine
)

Base = declarative_base()
```

10.1 Explicacao

- `create_engine` cria a conexao com o PostgreSQL.
- `SessionLocal` cria sessoes de acesso ao banco. Cada requisicao abrirea e fechara uma sessao.
- `Base` sera herdada pelas classes que representam tabelas.

11. Programar models.py

Este arquivo define as tabelas do banco. O SQLAlchemy usa classes Python para representar tabelas.

```
from sqlalchemy import Column, Integer, String, Boolean, ForeignKey, DateTime, Numeric
from sqlalchemy.orm import relationship
from sqlalchemy.sql import func

from database import Base

class Empresa(Base):
    __tablename__ = "empresa"

    id_empresa = Column(Integer, primary_key=True, index=True)
    razao_social = Column(String(150), nullable=False)
    nome_fantasia = Column(String(150))
    cnpj = Column(String(14), nullable=False, unique=True, index=True)
    email = Column(String(150), nullable=False, unique=True)
    telefone = Column(String(20))
    senha_hash = Column(String(255), nullable=False)
    status = Column(String(20), default="ATIVA")
    data_cadastro = Column(DateTime, server_default=func.now())

    perfis = relationship("EmpresaPerfil", back_populates="empresa")
    enderecos = relationship("Endereco", back_populates="empresa")

class Perfil(Base):
```

```

__tablename__ = "perfil"

id_perfil = Column(Integer, primary_key=True, index=True)
nome = Column(String(50), nullable=False, unique=True)
descricao = Column(String(255))

empresas = relationship("EmpresaPerfil", back_populates="perfil")

class EmpresaPerfil(Base):
    __tablename__ = "empresa_perfil"

    id_empresa_perfil = Column(Integer, primary_key=True, index=True)
    id_empresa = Column(Integer, ForeignKey("empresa.id_empresa"), nullable=False)
    id_perfil = Column(Integer, ForeignKey("perfil.id_perfil"), nullable=False)
    ativo = Column(Boolean, default=True)

    empresa = relationship("Empresa", back_populates="perfis")
    perfil = relationship("Perfil", back_populates="empresas")

class Endereco(Base):
    __tablename__ = "endereco"

    id_endereco = Column(Integer, primary_key=True, index=True)
    id_empresa = Column(Integer, ForeignKey("empresa.id_empresa"), nullable=False)
    tipo = Column(String(30), default="MATRIZ")
    logradouro = Column(String(150))
    numero = Column(String(20))
    bairro = Column(String(100))
    cidade = Column(String(100), nullable=False)
    estado = Column(String(2), nullable=False)
    cep = Column(String(8))
    latitude = Column(Numeric(10, 8))
    longitude = Column(Numeric(11, 8))

    empresa = relationship("Empresa", back_populates="enderecos")

```

11.1 Explicacao do modelo

- Empresa armazena CNPJ, razao social, email, telefone, senha criptografada e status.
- Perfil guarda os papeis existentes: FORNECEDOR, COMPRADOR e TRANSPORTADORA.
- EmpresaPerfil faz a ligacao entre empresa e perfil. Uma empresa pode ter mais de um perfil.
- Endereco armazena a localizacao da empresa. Essa informacao sera importante depois para frete e logistica.

12. Programar schemas.py

Schemas definem o formato dos dados que entram e saem da API. Eles tambem aparecem automaticamente no Swagger.

```

from pydantic import BaseModel, EmailStr
from typing import Optional

```

```

class EnderecoCreate(BaseModel):
    tipo: Optional[str] = "MATRIZ"
    logradouro: Optional[str] = None
    numero: Optional[str] = None
    bairro: Optional[str] = None
    cidade: str
    estado: str
    cep: Optional[str] = None

```

```

class EnderecoResponse(BaseModel):
    id_endereco: int
    tipo: Optional[str]
    logradouro: Optional[str]
    numero: Optional[str]
    bairro: Optional[str]
    cidade: str
    estado: str
    cep: Optional[str]

```

```

class Config:
    from_attributes = True

```

```
class FornecedorCreate(BaseModel):
    razao_social: str
    nome_fantasia: Optional[str] = None
    cnpj: str
    email: EmailStr
    telefone: Optional[str] = None
    senha: str
    endereco: EnderecoCreate
```

```
class FornecedorUpdate(BaseModel):
    razao_social: Optional[str] = None
    nome_fantasia: Optional[str] = None
    email: Optional[EmailStr] = None
    telefone: Optional[str] = None
    status: Optional[str] = None
```

```
class FornecedorResponse(BaseModel):
    id_empresa: int
    razao_social: str
    nome_fantasia: Optional[str]
    cnpj: str
    email: str
    telefone: Optional[str]
    status: str
```

```
class Config:
    from_attributes = True
```

12.1 Explicacao

- FornecedorCreate e usado no cadastro.
- FornecedorUpdate e usado na atualizacao. Como os campos sao opcionais, o aluno pode atualizar apenas o que quiser.
- FornecedorResponse controla o que sera devolvido pela API. A senha nunca deve ser retornada.
- EmailStr valida se o email possui formato valido.

13. Programar fornecedor_routes.py

Este arquivo contem o CRUD. Aqui ficam os endpoints da API.

```
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from passlib.context import CryptContext

from database import SessionLocal
from models import Empresa, Perfil, EmpresaPerfil, Endereco
from schemas import FornecedorCreate, FornecedorUpdate, FornecedorResponse

router = APIRouter(
    prefix="/fornecedores",
    tags=["Fornecedores"]
)

pwd_context = CryptContext(
    schemes=["bcrypt"],
    deprecated="auto"
)

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

def gerar_hash_senha(senha: str) -> str:
    return pwd_context.hash(senha)

def obter_perfil_fornecedor(db: Session):
    perfil = db.query(Perfil).filter(
        Perfil.nome == "FORNECEDOR"
    ).first()
```

```

if not perfil:
    perfil = Perfil(
        nome="FORNECEDOR",
        descricao="Empresa que fornece produtos"
    )

    db.add(perfil)
    db.commit()
    db.refresh(perfil)

return perfil

```

13.1 Criar fornecedor - POST

```

@router.post(
    "/",
    response_model=FornecedorResponse,
    summary="Cadastrar fornecedor",
    description="Cadastra uma empresa e associa o perfil FORNECEDOR."
)
def criar_fornecedor(
    dados: FornecedorCreate,
    db: Session = Depends(get_db)
):
    empresa_existente = db.query(Empresa).filter(
        Empresa.cnpj == dados.cnpj
    ).first()

    if empresa_existente:
        raise HTTPException(
            status_code=400,
            detail="Ja existe uma empresa cadastrada com este CNPJ."
        )

    email_existente = db.query(Empresa).filter(
        Empresa.email == dados.email
    ).first()

    if email_existente:
        raise HTTPException(
            status_code=400,
            detail="Ja existe uma empresa cadastrada com este e-mail."
        )

    empresa = Empresa(
        razao_social=dados.razao_social,
        nome_fantasia=dados.nome_fantasia,
        cnpj=dados.cnpj,
        email=dados.email,
        telefone=dados.telefone,
        senha_hash=gerar_hash_senha(dados.senha),
        status="ATIVA"
    )

    db.add(empresa)
    db.commit()
    db.refresh(empresa)

    perfil_fornecedor = obter_perfil_fornecedor(db)

    empresa_perfil = EmpresaPerfil(
        id_empresa=empresa.id_empresa,
        id_perfil=perfil_fornecedor.id_perfil,
        ativo=True
    )

    db.add(empresa_perfil)

    endereco = Endereco(
        id_empresa=empresa.id_empresa,
        tipo=dados.endereco.tipo,
        logradouro=dados.endereco.logradouro,
        numero=dados.endereco.numero,
        bairro=dados.endereco.bairro,
        cidade=dados.endereco.cidade,
        estado=dados.endereco.estado,
        cep=dados.endereco.cep
    )

    db.add(endereco)
    db.commit()
    db.refresh(empresa)

```

```
return empresa
```

13.2 Listar fornecedores - GET

```
@router.get(
    "/",
    response_model=list[FornecedorResponse],
    summary="Listar fornecedores",
    description="Lista todas as empresas com perfil FORNECEDOR."
)
def listar_fornecedores(db: Session = Depends(get_db)):
    fornecedores = (
        db.query(Empresa)
        .join(EmpresaPerfil)
        .join(Perfil)
        .filter(
            Perfil.nome == "FORNECEDOR",
            EmpresaPerfil.ativo == True
        )
        .all()
    )

    return fornecedores
```

13.3 Buscar fornecedor por ID - GET

```
@router.get(
    "/{id_empresa}",
    response_model=FornecedorResponse,
    summary="Buscar fornecedor por ID",
    description="Busca uma empresa cadastrada com perfil FORNECEDOR."
)
def buscar_fornecedor(
    id_empresa: int,
    db: Session = Depends(get_db)
):
    fornecedor = (
        db.query(Empresa)
        .join(EmpresaPerfil)
        .join(Perfil)
        .filter(
            Empresa.id_empresa == id_empresa,
            Perfil.nome == "FORNECEDOR",
            EmpresaPerfil.ativo == True
        )
        .first()
    )

    if not fornecedor:
        raise HTTPException(
            status_code=404,
            detail="Fornecedor nao encontrado."
        )

    return fornecedor
```

13.4 Atualizar fornecedor - PUT

```
@router.put(
    "/{id_empresa}",
    response_model=FornecedorResponse,
    summary="Atualizar fornecedor",
    description="Atualiza dados cadastrais de uma empresa fornecedora."
)
def atualizar_fornecedor(
    id_empresa: int,
    dados: FornecedorUpdate,
    db: Session = Depends(get_db)
):
    fornecedor = (
        db.query(Empresa)
        .join(EmpresaPerfil)
        .join(Perfil)
        .filter(
            Empresa.id_empresa == id_empresa,
            Perfil.nome == "FORNECEDOR",
            EmpresaPerfil.ativo == True
        )
        .first()
    )

    if not fornecedor:
```



```

        raise HTTPException(
            status_code=404,
            detail="Fornecedor nao encontrado."
        )

    dados_atualizacao = dados.model_dump(exclude_unset=True)

    for campo, valor in dados_atualizacao.items():
        setattr(fornecedor, campo, valor)

    db.commit()
    db.refresh(fornecedor)

    return fornecedor

```

13.5 Inativar fornecedor - DELETE

```

@router.delete(
   ("/{id_empresa}",
    summary="Inativar fornecedor",
    description="Inativa a empresa fornecedora sem remover fisicamente do banco."
)
def excluir_fornecedor(
    id_empresa: int,
    db: Session = Depends(get_db)
):
    fornecedor = (
        db.query(Empresa)
        .join(EmpresaPerfil)
        .join(Perfil)
        .filter(
            Empresa.id_empresa == id_empresa,
            Perfil.nome == "FORNECEDOR",
            EmpresaPerfil.ativo == True
        )
        .first()
    )

    if not fornecedor:
        raise HTTPException(
            status_code=404,
            detail="Fornecedor nao encontrado."
        )

    fornecedor.status = "INATIVA"

    db.commit()

    return {
        "mensagem": "Fornecedor inativado com sucesso."
    }

```

14. Programar main.py

Este arquivo inicializa a aplicacao FastAPI, cria as tabelas no banco, configura CORS e carrega as rotas de fornecedores.

```

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from database import Base, engine
from fornecedor_routes import router as fornecedor_router

Base.metadata.create_all(bind=engine)

app = FastAPI(
    title="Portal B2B - API de Fornecedores",
    description="Microservico responsavel pelo CRUD de fornecedores.",
    version="1.0.0"
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(fornecedor_router)

```

```
@app.get(
    "/",
    summary="Status da API",
    description="Verifica se a API esta ativa."
)
def home():
    return {
        "mensagem": "API de Fornecedores ativa."
    }
```

15. Executar o backend

No terminal, dentro da pasta fornecedor-service e com o ambiente virtual ativo, execute:

```
uvicorn main:app --reload
```

Se tudo estiver correto, o terminal mostrara que a aplicacao esta rodando em <http://127.0.0.1:8000>. O parametro `--reload` reinicia o servidor automaticamente quando algum arquivo e alterado.

16. Acessar o Swagger

Abra o navegador em:

```
http://localhost:8000/docs
```

O Swagger mostrara todos os endpoints automaticamente. Isso acontece porque o FastAPI gera a documentacao OpenAPI com base nas rotas e schemas do projeto.

Tambem existe a documentacao alternativa:

```
http://localhost:8000/redoc
```

17. Testar o cadastro de fornecedor

No Swagger, abra o endpoint POST `/fornecedores/` e clique em Try it out. Use o JSON abaixo.

```
{
  "razao_social": "Fornecedor Alpha LTDA",
  "nome_fantasia": "Alpha Alimentos",
  "cnpj": "12345678000199",
  "email": "contato@alpha.com",
  "telefone": "62999998888",
  "senha": "123456",
  "endereco": {
    "tipo": "MATRIZ",
    "logradouro": "Rua 10",
    "numero": "100",
    "bairro": "Centro",
    "cidade": "Goiania",
    "estado": "GO",
    "cep": "74000000"
  }
}
```

Se a resposta tiver status 200 ou 201 e retornar os dados do fornecedor, o cadastro funcionou.

18. Testar os demais endpoints

Operacao	Como testar
Listar	GET <code>/fornecedores/</code> - deve retornar uma lista de fornecedores.
Buscar por ID	GET <code>/fornecedores/{id_empresa}</code> - informe o ID retornado no cadastro.
Atualizar	PUT <code>/fornecedores/{id_empresa}</code> - envie apenas os campos que deseja alterar.
Inativar	DELETE <code>/fornecedores/{id_empresa}</code> - muda status para INATIVA.

18.1 Exemplo de atualizacao

```
{
  "nome_fantasia": "Alpha Distribuidora",
  "telefone": "62988887777"
}
```

}

19. Verificar os dados no PostgreSQL

Depois de cadastrar, confira se as tabelas foram preenchidas.

```
SELECT * FROM empresa;  
SELECT * FROM perfil;  
SELECT * FROM empresa_perfil;  
SELECT * FROM endereco;
```

A tabela empresa deve conter a empresa cadastrada. A tabela perfil deve conter FORNECEDOR. A tabela empresa_perfil deve ligar a empresa ao perfil. A tabela endereco deve conter o endereco informado.

20. O que cada aluno deve entender

- A API não grava fornecedor em uma tabela fornecedor, porque fornecedor é um papel da empresa.
- O CNPJ pertence a empresa, não ao perfil.
- A senha deve ser gravada como hash, nunca em texto puro.
- O Swagger é gerado automaticamente pelo FastAPI.
- O SQLAlchemy cria as tabelas a partir das classes definidas em models.py.
- Os schemas do Pydantic validam a entrada antes de chegar ao banco.
- A exclusão lógica evita perda de histórico.

21. Erros comuns e como corrigir

Erro	Possível causa e correção
psycopg2.OperationalError	Banco não está rodando, usuário/senha errados ou DATABASE_URL incorreta.
ModuleNotFoundError	Dependência não instalada ou ambiente virtual não ativado. Ative o venv e rode pip install.
EmailStr error	Instale email-validator. Comando: pip install email-validator.
Address already in use	Porta 8000 ocupada. Pare o servidor anterior ou use uvicorn main:app --reload --port 8001.
CNPJ duplicado	A regra UNIQUE funcionou. Use outro CNPJ ou apague o registro de teste.
Swagger não abre	Confira se o uvicorn está rodando e se a URL está correta: /docs.

22. Checklist de entrega do aluno

- Banco portal_b2b criado no PostgreSQL.
- Ambiente virtual criado e ativo.
- Dependências instaladas.
- Arquivos database.py, models.py, schemas.py, fornecedor_routes.py e main.py criados.
- API executando sem erro com uvicorn.
- Swagger abrindo em <http://localhost:8000/docs>.
- Cadastro de fornecedor funcionando.
- Listagem, busca, atualização e inativação funcionando.
- Dados conferidos no PostgreSQL.
- requirements.txt gerado.

23. Padrão para os demais microsserviços

Os outros microsserviços devem seguir o mesmo padrão estrutural. Troca-se o domínio, mas a organização permanece.

database.py	-> conexao com banco
models.py	-> tabelas do microservico
schemas.py	-> entradas e respostas
routes.py	-> endpoints
main.py	-> inicializacao da API

Exemplo: o microservico de produtos tera Produto, Categoria, UnidadeMedida e TipoTransporte. O microservico de demanda tera Demanda e DemandaRecorrencia. O padrao de construcao e o mesmo.

24. Evolucao futura: React e Kafka

Depois que o CRUD estiver funcionando no Swagger, a proxima etapa sera criar um front-end React para consumir a API. Depois disso, sera adicionada a publicacao de eventos Kafka.

```
POST /fornecedores/  
  -> grava no PostgreSQL  
  -> futuramente publica evento empresa_cadastrada no Kafka  
  -> outros microservicos consomem o evento
```

Conclusao

Ao final deste laboratorio, o aluno tera um microservico funcional, documentado no Swagger, persistindo dados em PostgreSQL e pronto para evoluir para front-end React e comunicacao assíncrona com Kafka.